IT ინფრასტრუქტურის მოთხოვნები — NAPR ხარისხის კონტროლის პლატფორმა

> **ვისთვის:** ჩემთვის, სალომესთვის და, საჭიროებისას, NAPR-ის IT სამსახურისთვის — სამუშაო ჩამონათვალი შეხვედრაზე. **მდგომარეობა:** 2026-10-22. ავსებს და აზუსტებს ადრინდელ `docs/IT_REQUEST.md` -ს (2026-10-20). **სტატუსის ნიშნები:** ✅ დადასტურებული · 🗓 დაგეგმილი · 💡 რეკომენდებული · ❓ IT-ის დასადასტურებელი · ⚠️ შეუსაბამობა. **რიცხვები:** CPU, RAM და დისკის ყოველი ციფრი **შეფასებაა** 💡 (არა გაზომვა), და მიზეზი თან ახლავს. დატვირთვის ტესტი ჯერ არ ჩატარებულა.

---

0. როგორ არის აწყობილი

თითო მოთხოვნას (R1–R26) აქვს:

1. **რა გვჭირდება** — ტექნიკურად;
2. **რატომ** — ტექნიკური მიზეზი;
3. **რა ფუნქციაა დამოკიდებული**;
4. **სავალდებულოა თუ არჩევითი** და რომელ ფაზაში;
5. **ალტერნატივები**;
6. **რა მოხდება, თუ IT ვერ მოგვცემს**;
7. 🗣 **შეხვედრაზე მარტივად** — როგორ ავხსნა ხმამაღლა.

სავალდებულოობა:

- **M** — სავალდებულო: ამის გარეშე ფაზა ვერ დაიწყება;
- **S** — სასურველი: ალტერნატივით შეიძლება;
- **O** — არჩევითი.

ფაზები:

- **P1** — პილოტი;
- **P2** — წყაროები;
- **P3** — სრული გაშვება;
- **P4** — Pre-QC.

ზოგადი პრინციპები, რომლებიც ყველა მოთხოვნას ეხება:

1. **მხოლოდ კითხვა** — სააგენტოს სისტემებში (REGFIELDALL, REDI, SLR) არაფერს ვწერთ.
2. **საკუთარი ასლი** — საჭირო ველების ასლს ვინახავთ და განრიგით ვანახლებთ, ამიტომ წყაროს დატვირთვა მცირეა.

3. **მინიმალური მონაცემი** — მხოლოდ ის ველები, რაც სამუშაოს სჭირდება.
4. **მინიმალური უფლება** — ცალკე სერვისის ანგარიში თითო წყაროზე, მხოლოდ ჩვენი სერვერიდან.
5. **გადატანადობა** — ყველა პარამეტრი კონფიგურაციაშია (`QCP_*` გარემოს ცვლადები), კოდში სერვერი არ წერია.

---

1. შემაჯამებელი ჩამონათვალი

ID	მოთხოვნა	M/S/O	ფაზა	ვისზეა	✓
R1	ვირტუალური სერვერი (VM)	M	P1	IT	☐
R2	OS და runtime (Python ≥ 3.12 ან Docker)	M	P1	IT + ჩვენ	☐
R3	PostgreSQL ბაზა (ცალკე ბაზა + 2 მომხმარებელი)	M	P1	IT (DBA)	☐
R4	ბაზის ბექაპი (P1: ღამის; P3: PITR)	M	P1/P3	IT	☐
R5	ფაილების საცავი (ფოტო/PDF) + ბექაპი	M	P1	IT	☐
R6	დომენი (ქვედომენი) და DNS	M	P1	IT	☐
R7	TLS სერტიფიკატი	M	P1	IT	☐
R8	Reverse proxy / load balancer	M	P1	IT	☐
R9	წვდომა შიდა ქსელიდან	M	P1	IT	☐
R10	წვდომა გარედან (ველი, ტელეფონი) — საჯარო HTTPS ან VPN	M (P3), S (P1)	P1/P3	IT + უსაფრთხოება	☐
R11	სერვერიდან გამავალი კავშირი (Google OAuth, SMTP, პაკეტები)	M	P1	IT	☐
R12	Google Cloud პროექტი + OAuth client (Web)	M	P1	IT	☐
R13	SMTP relay + გამგზავნი მისამართი (SPF/DKIM)	S (P1), M (P3)	P1/P3	IT	☐
R14	REGFIELDALL-ზე წვდომა (view ან ექსპორტი) + სერვისის ანგარიში	M	P2	IT / მონაცემის მფლობელი	☐
R15	სპორადულის სია (view ან SFTP), განმცხადებლების მონაცემის გარეშე	S	P2	IT	☐
R16	ოქმის დეტალების API (შემსრულებლები, დანართები)	O (P2), S (P4)	P4	IT	☐
R17	REDI / SLR / WFS — სერვისის ანგარიში (Pre-QC)	S	P4	IT	☐

ID	მოთხოვნა	M/S/O	ფაზა	ვისზეა	✓
R18	თანამშრომლების მონაცემი: გუნდები, ახალი/წასული (HR/AD)	S	P3	HR + IT	☐
R19	საიდუმლოებების შენახვა	M	P1	IT	☐
R20	მონიტორინგი, ლოგები, განგაში	S (P1), M (P3)	P1/P3	IT	☐
R21	სატესტო გარემო (staging)	S	P1	IT	☐
R22	ვერსიის კონტროლი (git) + deployment-ის წვდომა	M	P1	IT + ჩვენ	☐
R23	უსაფრთხოების შემოწმება, პერსონალური მონაცემის შეფასება, შენახვის ვადები	M (P3)	P3	IT-უსაფრთხოება + იურიდიული	☐
R24	დრო და სასაათო სარტყელი (NTP, Asia/Tbilisi)	M	P1	IT	☐
R25	ოპერაციული პასუხისმგებლობა (RACI), ინციდენტის პროცესი	M	P1	IT + ჩვენ	☐
R26	ატვირთული ფაილების ანტივირუსული შემოწმება	O	P3	IT	☐

პილოტისთვის (P1) მინიმუმი: R1, R2, R3, R4 (ღამის), R5, R6, R7, R8, R9, R11, R12, R19, R22, R24, R25. **მარტივად:** პილოტისთვის „ოთახი, კარადა, მისამართი, დაცული კარი და შესასვლელი“ გვჭირდება. მონაცემის წყაროები პილოტში ჯერ დღევანდელი გზით მოვა.

---

2. თითო მოთხოვნა დეტალურად

R1. ვირტუალური სერვერი (VM)

1. **რა გვჭირდება:** ერთი VM. 💡 საწყისი შეფასება: **4 vCPU, 8 GB RAM, 100 GB SSD**, PostgreSQL-ის ჩათვლით, თუ ბაზა იმავე VM-ზეა. ბაზის ცალკე სერვერზე განთავსებისას აპლიკაციის VM-ს 2 vCPU და 4 GB RAM ეყოფა (შეფასება).
2. **რატომ:** აპლიკაცია (uvicorn, 2–4 worker), ფონური ამოცანები (ღამის იმპორტი, მეილის რიგი), reverse proxy (თუ VM-ზეა) და PostgreSQL ერთად. შეფასების საფუძველი:
   - ≈ 300 მომხმარებელი, ერთდროულად ≈ 40 (`IT_REQUEST.md` -ის შეფასება);
   - ✅ დღევანდელი ბაზა 94 MB-ია, 44 159 შემთხვევაზე;
   - ✅ სტატისტიკა SQLite-ზე 2–3 წმ-ს იღებს;
   - ✅ REGFIELDALL-ის ასლი ≈ 1 GB-ია.
3. **დამოკიდებულია:** ყველაფერი.
4. **M, P1.**

5. **ალტერნატივები:**
    - ადგილი არსებულ აპლიკაციის სერვერზე ან კონტეინერის პლატფორმაზე (Docker host, Kubernetes namespace);
    - მცირე რესურსით დაწყება და გაზრდა გაზომვის შემდეგ.
6. **თუ ვერ მოგვცემს:** პლატფორმა ვერ გაიშვება. დარჩება პროტოტიპი ერთ კომპიუტერზე, რაც საწარმოოსთვის მიუღებელია (სახელმძღვანელო A.2).
7. 🗣 **მარტივად:** „ერთი ვირტუალური სერვერი გვჭირდება, საშუალო ოფისის კომპიუტერის სიმძლავრის. ეს შეფასებაა: პილოტზე გავზომავთ და საჭიროებისას შევამცირებთ ან გავზრდით.“

R2. ოპერაციული სისტემა და runtime

1. **რა გვჭირდება:** Linux (Ubuntu LTS / RHEL-ის ოჯახი) ან Windows Server. Python ≥ 3.12, ვირტუალური გარემოთი; ან Docker Engine, თუ კონტეინერი აირჩევა. სააგენტოს root CA-ს სერტიფიკატი OS-ის საცავში (კორპორატიული TLS-ისთვის). Linux-ზე: systemd სერვისი; Windows-ზე: Windows Service ან კონტეინერი.
2. **რატომ:**
    - ✅ კოდი OS-დამოუკიდებელია (`pathlib` , `QCP_*`);
    - ✅ `truststore` OS-ის სერტიფიკატების საცავს იყენებს, ამიტომ შიდა CA იქ უნდა იყოს. თორემ Google OAuth-ზე და SMTP TLS-ზე კავშირი ჩავარდება;
    - ✅ ტესტირებულია Python 3.14.5-ზე, მოთხოვნა ≥ 3.12.
3. **დამოკიდებულია:** აპლიკაციის გაშვება.
4. **M, P1.**
5. **ალტერნატივები:** ჩვენ ვამზადებთ Docker image-ს, IT კი მხოლოდ კონტეინერის host-ს გვაძლევს. ⚠️ Dockerfile ჯერ არ არსებობს — ეს ჩვენი სამუშაოა.
6. **თუ ვერ მოგვცემს:** —
7. 🗣 **მარტივად:** „თქვენი სტანდარტული ოპერაციული სისტემა გამოგვადგება. პროგრამა ორივეზე მუშაობს. Docker-ის „ყუთში“ თუ მოგაწვდით, ინსტალაციაც მარტივი იქნება.“

R3. PostgreSQL ბაზა

1. **რა გვჭირდება:** PostgreSQL 14+ (💡 სასურველია 16), ცალკე ბაზა (მაგ. `qc`), UTF-8 კოდირებით. ორი მომხმარებელი:
    - `qc_app` — აპლიკაცია: მონაცემის წაკითხვა და ჩაწერა, სქემის შეცვლის გარეშე;
    - `qc_migrator` — სქემის მფლობელი, მიგრაციისთვის. superuser არ გვჭირდება. კავშირი მხოლოდ აპლიკაციის სერვერიდან; TLS, თუ ბაზა სხვა VM-ზეა.
2. **რატომ:**
    - ერთდროული ჩაწერა (MVCC);
    - ტრანზაქციები და შეზღუდვები;
    - ბექაპი / PITR;
    - უფლებების მართვა. SQLite (✅ პროტოტიპი) ერთ მწერალს უშვებს და ქსელურ წვდომას არ იძლევა (სახელმძღვანელო B.11).

3. **დამოკიდებულია:** ყველა მონაცემი — ოქმები, ისტორია, მომხმარებლები, სესიები.
4. **M, P1.**
5. **ალტერნატივები:** Microsoft SQL Server, თუ ეს სააგენტოს სავალდებულო სტანდარტია. ⚠️ ამ შემთხვევაში ჩვენ ტესტირება და დიალექტის განსხვავებების გასწორება გვიწევს (JSON სვეტები, collation, თარიღები).
6. **თუ ვერ მოგვცემს:** სერვერული ბაზის გარეშე საწარმოო გაშვება შეუძლებელია.
7. 🗣 **მარტივად:** „ბაზა ცენტრალური, დაცული „კარადაა“, სადაც საიტი ყველა ჩანაწერს ინახავს. თქვენს არსებულ DB სერვერზე ცალკე ბაზაც საკმარისია. ახალი სერვერი სავალდებულო არ არის.“

⚠️ **ჩვენი მხრიდან (G1, G2):** PostgreSQL-ის დრაივერი ჯერ არ არის დაყენებული და ტესტი არ ჩატარებულა; მიგრაციის ინსტრუმენტი (Alembic) დასამატებელია. 💡 სატესტო PostgreSQL ადრე რომ მივიღოთ, ამ სამუშაოს პარალელურად გავაკეთებთ.

R4. ბაზის ბექაპი და აღდგენა

1. **რა გვჭირდება:**
   - **P1:** ღამის სრული ბექაპი (`pg_dump -Fc` ან თქვენი ინსტრუმენტი), მინიმუმ 14 დღის შენახვით;
   - **P3:** PITR — base backup + WAL-ის არქივი (pgBackRest / Barman / თქვენი ინსტრუმენტი);
   - ბექაპი **სხვა მედიაზე ან ლოკაციაზე** უნდა იყოს და დაშიფრული;
   - აღდგენის ტესტი კვარტალში ერთხელ.
2. **რატომ:** QC-ის ისტორია და გადაწყვეტილებები ორგანიზაციის ჩანაწერებია. 💡 შეთავაზება: RPO ≤ 15 წთ, RTO ≤ 4 სთ (❓ ხელმძღვანელობისა და IT-ის დასამტკიცებელი).
3. **დამოკიდებულია:** მონაცემის დაკარგვისგან დაცვა.
4. **M:** P1 — ღამის; P3 — PITR.
5. **ალტერნატივები:**
   - VM-ის სნეპშოტი. მარტივია, მაგრამ შეიძლება არათანმიმდევრული აღმოჩნდეს, თუ ბაზა ჩაწერის შუაშია. ბაზის ბექაპის ნაცვლად არ გამოდგება;
   - მართული PostgreSQL სერვისი, რომელსაც ბექაპი ჩაშენებული აქვს.
6. **თუ ვერ მოგვცემს:** დისკის ერთი დაზიანება ყველაფერს წაშლის. ✅ დღეს ბექაპი არ კეთდება, ხოლო `build_db.py` აღდგენა არ არის (ბაზას ნულიდან აწყობს).
7. 🗣 **მარტივად:** „ყოველ ღამე ბაზის ასლი გვჭირდება, სხვა ადგილას. სრული გაშვებისთვის კი „ვიდეოჩანაწერი“, რომ ნებისმიერ წუთზე დავბრუნდეთ. და დროდადრო შევამოწმოთ, რომ ასლი მართლა აღდგება.“

R5. ფაილების საცავი (ფოტო / PDF)

1. **რა გვჭირდება:** დისკზე ან ქსელურ საცავზე საქაღალდე (✅ დღეს `data/uploads/<item_id>/`), აპლიკაციის მომხმარებლის ჩაწერის უფლებით, და ბექაპი

ბაზის ბექაპის იმავე განრიგით. 💡 მოცულობა: 1–3 GB წელიწადში (`IT_REQUEST.md`-ის შეფასება; ✅ დღეს 0 ატვირთვაა ბაზაში).
2. **რატომ:** ✅ ფოტოს (JPEG/PNG/WEBP/GIF/PDF, ≤ 10 MB) ტვირთავს მხოლოდ კონტროლი, და ის ოქმის ისტორიის ნაწილია. ბაზაში მხოლოდ მეტამონაცემი ინახება (`attachments`).
3. **დამოკიდებულია:** ფოტოს ატვირთვა და ნახვა ბარათზე და მეილში.
4. **M, P1.**
5. **ალტერნატივები:** ობიექტური საცავი (S3-თავსებადი). 💡 ამისთვის კოდის მცირე ცვლილებაა საჭირო.
6. **თუ ვერ მოგვცემს:** ფოტოები VM-ის ლოკალურ დისკზე დარჩება, ბექაპის გარეშე. VM-ის ხელახლა აწყობისას ისინი დაიკარგება.
7. 🗣 **მარტივად:** „ფოტოებს ცალკე „ალბომი“ სჭირდება, და ალბომის ასლიც, ბაზის ასლთან ერთად.“

R6. დომენი და DNS

1. **რა გვჭირდება:** ქვედომენი, მაგ. `qc.<სააგენტოს დომენი>` (❓ სახელი IT-ის წესით). A ან CNAME ჩანაწერი შიდა DNS-ში და, გარე წვდომის შემთხვევაში (R10), საჯარო DNS-შიც.
2. **რატომ:**
   - ✅ Google OAuth-ის redirect URI (`<QCP_BASE_URL>/auth/google/callback`) სტაბილურ, HTTPS მისამართს მოითხოვს;
   - ✅ მეილის ბმულები `QCP_BASE_URL`-ზეა აგებული.
3. **დამოკიდებულია:** შესვლა, მეილის ბმულები, მომხმარებლების წვდომა.
4. **M, P1.**
5. **ალტერნატივები:** არსებული დომენის ქვებილიკი (`https://<დომენი>/qc`). ⚠️ ამას proxy-ს და აპლიკაციის კონფიგურაციის (root path) ცვლილება სჭირდება — 💡 შესამოწმებელია.
6. **თუ ვერ მოგვცემს:** Google-ით შესვლა ვერ ჩაირთვება. IP მისამართით მუშაობა არასანდოა.
7. 🗣 **მარტივად:** „საიტს სახელი სჭირდება, IP-ის ნაცვლად, რომ Google-მა „იცოდეს“, სად დააბრუნოს თანამშრომელი შესვლის შემდეგ.“

R7. TLS სერტიფიკატი

1. **რა გვჭირდება:** სერტიფიკატი R6-ის დომენზე, TLS 1.2+, ავტომატური განახლებით ან ვადის კონტროლით. თუ ველიდან ტელეფონებით საჯარო წვდომა იქნება, 💡 საჯარო CA-ს სერტიფიკატი, რომ ტელეფონებზე შიდა root CA-ს დაყენება არ დაგვჭირდეს.
2. **რატომ:**
   - HTTPS Google OAuth-ის წინაპირობაა;
   - სესიის cookie და QC მონაცემი დაშიფრული უნდა მოძრაობდეს;
   - ✅ cookie-ს `Secure` ნიშანი HTTPS-ზე მუშაობს.
3. **დამოკიდებულია:** შესვლა, ყველა გვერდი.
4. **M, P1.**

5. **ალტერნატივები:** შიდა CA, თუ წვდომა მხოლოდ შიდაა ან მართულ მოწყობილობებზე.
6. **თუ ვერ მოგვცემს:** Google-ით შესვლა ვერ იმუშავებს, და მონაცემი ღიად მოძრაობს.
7. 🗣 **მარტივად:** „სერტიფიკატი საიტის „პირადობის მოწმობაა“ და „დალუქული კონვერტი“. მის გარეშე Google შესვლას არ დაუშვებს.“

R8. Reverse proxy / load balancer

1. **რა გვჭირდება:**
   - HTTPS-ის მიღება (TLS termination) და გადაცემა აპლიკაციაზე (`http://127.0.0.1:8200` ან შიდა IP);
   - მოთხოვნის ზომა ≥ 12 MB (ფოტო ≤ 10 MB + ფორმა);
   - timeout ≥ 120 წმ (განაწილების გადახედვა ათასობით ჩანაწერზე. ❓ PostgreSQL-ზე გასაზომია);
   - `X-Forwarded-For` და `X-Forwarded-Proto` თავსართები;
   - HTTP → HTTPS გადამისამართება; HSTS;
   - წვდომის ლოგი **query string-ის გარეშე**, ან მისი ნიღბვით, რომ OAuth-ის `code` ლოგში არ ჩაიწეროს.
2. **რატომ:**
   - Python-ის აპლიკაციის სერვერი ინტერნეტზე პირდაპირ არ უნდა ჩანდეს;
   - ⚠️ G4: აპლიკაციას proxy-ს IP-ისა და თავსართების ცოდნა სჭირდება, `--proxy-headers --forwarded-allow-ips`-ისთვის. თორემ cookie `Secure`-ის გარეშე დარჩება, და აუდიტში არასწორი IP ჩაიწერება.
3. **დამოკიდებულია:** HTTPS, ატვირთვა, ლოგები.
4. **M, P1.**
5. **ალტერნატივები:**
   - nginx, ან IIS + ARR, ჩვენს VM-ზე;
   - სააგენტოს ცენტრალური load balancer / WAF.
6. **თუ ვერ მოგვცემს:** proxy ჩვენს VM-ზე დავაყენებთ (nginx), ოღონდ სერტიფიკატი თქვენგან გვჭირდება.
7. 🗣 **მარტივად:** „proxy შენობის „მიმღებია“: დაცული შესასვლელი, რომელიც სტუმარს სწორ ოთახში ატარებს. თქვენი სტანდარტული მიმღები გვირჩევნია.“

R9. წვდომა შიდა ქსელიდან

1. **რა გვჭირდება:** სააგენტოს შიდა ქსელიდან (ოფისები, QC) დომენზე 443 პორტით წვდომა.
2. **რატომ:** QC ოპერატორები, მენეჯერები და ადმინები ოფისიდან მუშაობენ.
3. **დამოკიდებულია:** ყველა ოფისის მომხმარებელი.
4. **M, P1.**
5. **ალტერნატივები:** —
6. **თუ ვერ მოგვცემს:** —
7. 🗣 **მარტივად:** „ოფისიდან საიტი ჩვეულებრივი მისამართით უნდა იხსნებოდეს.“

R10. წვდომა გარედან (ველი, ტელეფონი, პლანშეტი)

1. **რა გვჭირდება:** ერთ-ერთი:
   - **(ა)** საჯარო HTTPS გამოქვეყნება (proxy/WAF DMZ-ში → შიდა სერვერი);
   - **(ბ)** VPN მობილურ მოწყობილობებზე. 💡 დამატებითი დაცვა (ა)-სთვის: WAF-ის წესები, rate limiting, HSTS, საჭიროებისას გეო-შეზღუდვა.
2. **რატომ:** ≈ 250 ამზომველი და 14 საველე მენეჯერი ველის პასუხებს ტელეფონიდან და პლანშეტიდან აგზავნის. ✅ ინტერფეისი მობილურზე მუშაობს: ველის სია ბარათებად, შემოწმებულია `run_lan.bat`-ით.
3. **დამოკიდებულია:** ველის პასუხები, „არ ვეთანხმები“, მენეჯერის დადასტურება და Pre-QC-ის ნახვა ველიდან.
4. **S (P1** — პილოტი შეიძლება ოფისით დაიწყოს), **M (P3).**
5. **ალტერნატივები:** მხოლოდ შიდა ქსელი + ველის მომხმარებლები ოფისში ან VPN-ით. ⚠️ ეს ველის სამუშაოს შეანელებს.
6. **თუ ვერ მოგვცემს:** ველი ტელეფონიდან ვერ იმუშავებს, და პლატფორმის ერთი მთავარი სარგებელი იკარგება.
7. 🗣 **მარტივად:** „ამზომველები ველში არიან. ან დაცული საჯარო მისამართი გვჭირდება, ან VPN ყველა ტელეფონზე. ჩვენ პირველს გირჩევთ, Google-ის შესვლითა და მხოლოდ დარეგისტრირებული ელფოსტებით. მაგრამ გადაწყვეტილება თქვენი პოლიტიკისაა.“

R11. სერვერიდან გამავალი კავშირი

1. **რა გვჭირდება:**
   - `oauth2.googleapis.com:443` — ✅ token-ის გაცვლა (`google_auth.TOKEN_URL`). `accounts.google.com`-ზე მომხმარებლის ბრაუზერი გადადის და არა სერვერი;
   - SMTP relay (R13);
   - ინსტალაციის დროს: PyPI (`pypi.org` , `files.pythonhosted.org`), ან შიდა სარკე, ან მზა Docker image;
   - P4-ში: WFS (`wblr.napr.gov.ge:443`) და REDI-ს API, თუ გაჩნდება.
2. **რატომ:** OIDC-ის Authorization Code ნაკადში სერვერი თავად ეკითხება Google-ს token-ს.
3. **დამოკიდებულია:** Google-ით შესვლა, მეილი, ინსტალაცია.
4. **M, P1.**
5. **ალტერნატივები:**
   - კორპორატიული HTTP proxy-ით. 💡 `requests` `HTTPS_PROXY`-ს კითხულობს; ✅ `truststore` კორპორატიული სერტიფიკატისთვის;
   - ინსტალაციისთვის: ოფლაინ wheel-ები ან Docker image.
6. **თუ ვერ მოგვცემს:** Google-ით შესვლა ვერ იმუშავებს. საჭირო გახდება სხვა OIDC პროვაიდერი შიდა ქსელში (Entra ID / ADFS / Keycloak) — Q45.
7. 🗣 **მარტივად:** „სერვერმა Google-ს ერთი კითხვა უნდა დაუსვას შესვლისას: „ეს ადამიანი ნამდვილად ის არის?“. ამისთვის ერთ კონკრეტულ მისამართზე გასვლა სჭირდება, მეტი არაფერი.“

R12. Google Cloud პროექტი და OAuth client

1. **რა გვჭირდება:**
   - Google Cloud პროექტი **სააგენტოს სახელზე** (არა პირად ანგარიშზე), 💡 ცალკე პროექტი მხოლოდ ამ აპლიკაციისთვის;
   - OAuth consent screen: აპლიკაციის სახელი, დომენი, მხარდაჭერის ელფოსტა. ტიპი: Workspace-ის შემთხვევაში **Internal**, პირადი Gmail-ებისთვის **External** → სტატუსი „In production“ (Testing-ში მხოლოდ სატესტო მომხმარებლები შედიან) ❓;
   - OAuth client ID, ტიპი **Web application**;
   - Authorized redirect URI: `https://<დომენი>/auth/google/callback` (და სატესტო გარემოს მისამართი, თუ R21 იქნება);
   - scope: მხოლოდ `openid email profile`;
   - Client ID და Client Secret უსაფრთხო არხით, ან პირდაპირ სერვერის საიდუმლოების საცავში (R19).
2. **რატომ:** ✅ კოდი მზადაა (`app/routes/google_auth.py`) და ჩაირთვება, როცა `QCP_GOOGLE_CLIENT_ID` და `QCP_GOOGLE_CLIENT_SECRET` მიეთითება. ✅ არჩევით: `QCP_GOOGLE_ALLOWED_DOMAIN` — მხოლოდ Workspace დომენი.
3. **დამოკიდებულია:** საწარმოო შესვლა (დემო შესვლა საწარმოოში გაითიშება).
4. **M, P1.**
5. **ალტერნატივები:** Microsoft Entra ID / ADFS / Keycloak (OIDC), თუ ყველა თანამშრომელს სააგენტოს ანგარიში აქვს. 💡 მცირე ცვლილება კოდში.
6. **თუ ვერ მოგვცემს:** შესვლის საწარმოო მექანიზმი არ იქნება.
7. 🗣 **მარტივად:** „Google-ში საიტს „რეგისტრაცია“ სჭირდება, სააგენტოს სახელით. ჩვენ მხოლოდ სახელს და ელფოსტას ვითხოვთ. Gmail-ზე, Drive-ზე ან სხვა რამეზე წვდომა არ გვჭირდება.“

⚠️ **ჩვენი მხრიდან (G15):** ბაზაში 0 ელფოსტაა. მენეჯერები აგროვებენ, ადმინები კი შეიყვანენ, გაშვებამდე.

R13. SMTP relay და გამგზავნი მისამართი

1. **რა გვჭირდება:**
   - SMTP relay: host, port (✅ ნაგულისხმევი 587, STARTTLS), ავთენტიფიკაცია (თუ საჭიროა);
   - გამგზავნი მისამართი, მაგ. `qc-noreply@<დომენი>`, SPF/DKIM-ით (და DMARC-ით). 💡 მოცულობა: დღეში ათეულობით შეტყობინება, ძირითადად საველე მენეჯერებთან.
2. **რატომ:**
   - ✅ `services/notify.py`: შეტყობინება იგზავნება დაბრუნებაზე, QC მენეჯერის გადაწყვეტილებაზე და კომენტარზე, მხოლოდ ✉ მონიშვნით;
   - ✅ SMTP-ის გარეშე მეილი `preview` სტატუსით ინახება (ადმინისტრირება → მეილები).

3. **დამოკიდებულია:** შეტყობინებები მენეჯერებთან.
4. **S (P1), M (P3).**
5. **ალტერნატივები:** შეტყობინებები მხოლოდ პლატფორმაში (უკვე არსებობს: ფერები, რიცხვები, სიები).
6. **თუ ვერ მოგვცემს:** მენეჯერებს შეტყობინება მეილით არ მივა, და სისტემაში შესვლაზე იქნებიან დამოკიდებული.
7. 🗣 **მარტივად:** „მენეჯერს მეილი უნდა მივიდეს, როცა ოქმი დაუბრუნდა. სააგენტოს ფოსტით, რომ სპამში არ მოხვდეს.“

⚠️ **ჩვენი მხრიდან (G7):** გაგზავნა მოთხოვნის შიგნით ხდება. 💡 ფონურ რიგში გადავიტანთ.

R14. REGFIELDALL-ზე წვდომა

1. **რა გვჭირდება:** ერთ-ერთი (უპირატესობის რიგით):
   - **(ა)** read-only view: `FLD_PAPER_ID, CADCODE, MOX_CREATE_DATE, FLD_STATUS_NAME, OQMI_CREATOR, MOXAZ_CREATOR, IS_LAST`, ცალკე მომხმარებლით, რომელსაც მხოლოდ ამ view-ზე `SELECT` აქვს, მხოლოდ ჩვენი სერვერის IP-დან;
   - **(ბ)** REST API იგივე ველებით;
   - **(გ)** ღამის ექსპორტი სერვერისთვის ხელმისაწვდომ საქაღალდეში ან SFTP-ზე. ✅ დღეს არსებობს `Z:\REGFIELDALL\REGFIELDALL_<თარიღი>_01-00_<N>.gdb`.
     - (გ1) ამ საქაღალდეზე წაკითხვის უფლება სერვისის ანგარიშით (SMB 445) + GDAL სერვერზე;
     - (გ2) იგივე CSV-ად.
   - სიხშირე: ღამით + მოთხოვნით.
2. **რატომ:**
   - ✅ განაწილება კოდის **პირველ** ოქმს ეყრდნობა (უადრესი `MOX_CREATE_DATE`), „სისტემური სერვისის“ ოქმების გამორიცხვით და ღია საველე სტატუსის ფილტრით (`services/distribution/source.py`);
   - ✅ სპორადულზე ამზომველი MOXAZ_CREATOR-ით დგინდება;
   - ✅ წყარო ≈ 2.33 მლნ ჩანაწერია, ამიტომ ასლს ღამით ვანახლებთ.
3. **დამოკიდებულია:** სისტემური განაწილება; სპორადულის ამზომველების დადგენა; ძველი ოქმების ამზომველების შევსება (✅ 1 550 ოქმი).
4. **M, P2.** P1-ში შეიძლება დღევანდელი გზით: Distributor-ის ასლის ხელით გადატანა ან (გ1).
5. **ალტერნატივები:** ზემოთ (ა)–(გ). ❓ ვინ არის მონაცემის მფლობელი და ვინ ამტკიცებს წვდომას.
6. **თუ ვერ მოგვცემს:** განაწილება ჩვენი კომპიუტერიდან ხელით გადატანილ ასლზე დარჩება. ⚠️ ეს ერთი ადამიანის კომპიუტერზე დამოკიდებულებას ინარჩუნებს.
7. 🗣 **მარტივად:** „ყოველ ღამე საველე ოქმების სიიდან შვიდი სვეტი გვჭირდება, მხოლოდ საკითხავად. საუკეთესოა პატარა „ფანჯარა“ (view) თქვენს ბაზაში. თუ ეს არ შეიძლება, დღევანდელი ღამის ფაილიც გამოდგება, ოღონდ სერვერისთვის ხელმისაწვდომ ადგილას.“

R15. სპორადულის სია

1. **რა გვჭირდება:** view ან ყოველდღიური ფაილი (SFTP / დაცული საქაღალდე) ამ სვეტებით: `FLD_PAPER_ID, CADCODE, CREATE_DATE, SURVEY_DAY, SURVEY_TIME, CALL_STATUS, ST_NAME, TR_NAME, USER_REAL_NAME, REGNUMBER`. **განმცხადებლების მონაცემის გარეშე** (APPLICS, APP_PERSONS, REG_EMP, ADDRESS).
2. **რატომ:**
   - ✅ დღეს xlsx მეილით მოდის განმცხადებლების პირადი ნომრებითა და ტელეფონებით. პლატფორმა მათ არ კითხულობს და ფაილს შლის (`services/sporadic.py`), მაგრამ მეილით გავრცელება რისკია;
   - ✅ 20.10-ის ფაილი: 50 320 ჩანაწერი.
3. **დამოკიდებულია:** სპორადულის ყოველდღიური განაწილება.
4. **S, P2.**
5. **ალტერნატივები:** ✅ დღევანდელი მეილი + ხელით ატვირთვა (მუშაობს).
6. **თუ ვერ მოგვცემს:** ყოველ დილით ხელით ატვირთვა დარჩება, და პირადი მონაცემი ისევ მეილით გავრცელდება.
7. 🗣 **მარტივად:** „სპორადულის სია მეილის ნაცვლად დაცულ ადგილას გვჭირდება, და მხოლოდ საჭირო სვეტებით. სხვისი პირადი ნომრები და ტელეფონები არ გვჭირდება.“

R16. ოქმის დეტალების API

1. **რა გვჭირდება:** API: ოქმის ნომერი → JSON. ველები: შემსრულებლები სტატუსით „საველე სამუშაოების შესრულებაზე უფლებამოსილი პირი“ (სახელი, 💡 პირადი ნომერი მიბმისთვის), დანართების სია და Pre-QC-ისთვის საჭირო ველები. სერვისის ანგარიშით ან API გასაღებით.
2. **რატომ:**
   - ✅ დღეს ამზომველები OQMI_CREATOR / MOXAZ_CREATOR-დან ითვლება და სახელით ებმება Field Group-ს. რეგისტრატორი რომ ამზომველად არ ჩაითვალოს, FIELD_GROUP_ALL-ით იფილტრება;
   - 🗓 მომავალში „შემსრულებლები ოქმის API-დან“ (`OPEN_QUESTIONS.md`);
   - ✅ Pre-QC hosted-მა 31 რეალურ ოქმზე გაზომა: 49/49 ხელმოწერას პირადი ნომერი აქვს. ამით მიბმა სახელით კი არა, ცალსახად გახდებოდა.
3. **დამოკიდებულია:** ამზომველის ზუსტი მიბმა; Pre-QC სერვერზე.
4. **O (P2), S (P4).**
5. **ალტერნატივები:** ოქმის HTML/TXT ექსპორტი.
6. **თუ ვერ მოგვცემს:** სახელით მიბმა დარჩება (✅ მუშაობს ალიასებით და „დაუჯგუფებლით“), და Pre-QC ლოკალურად.
7. 🗣 **მარტივად:** „ოქმის „ვინ შეასრულა“ ზუსტად გვინდა, ოქმიდანვე. დღეს სახელებით ვადარებთ, და სახელები ზოგჯერ სხვადასხვანაირად იწერება.“

R17. REDI / SLR / WFS — სერვისის ანგარიში (Pre-QC)

1. **რა გვჭირდება:**
   - SLR WFS (`https://wblr.napr.gov.ge/data/SLR/ows`): სერვისის ანგარიში, მხოლოდ `GetFeature` , ფენები `SLR:GFLD_PARCELS` , `GFLD_BUILDINGS` , `GFLD_TOPOLINES` , `GFLD_EASEMENTS` ; სერვერის IP-ის დაშვება;
   - REDI-ზე: ოქმის მონაცემის API (R16), ან ოფიციალური გზა სერვერისთვის;
   - არჩევით: LR_BUILDINGS WMS.
2. **რატომ:**
   - ✅ Pre-QC-ის GIS-შემოწმებები WFS-ს იყენებს (`pre-qc-validator-master/app/wfs.py` , HTTP Basic, `.env` -ში შენახული მომხმარებლით);
   - ✅ REDI-ს გვერდებს დღეს ბრაუზერის გაფართოება კითხულობს, თანამშრომლის საკუთარი სესიით;
   - სერვერს ასეთი სესია არ აქვს, და მისი შექმნა სწორი არ არის.
3. **დამოკიდებულია:** Pre-QC-ის ავტომატური შემოწმება სერვერზე.
4. **S, P4.**
5. **ალტერნატივები:** Validator თანამშრომლის კომპიუტერზე რჩება და მხოლოდ შედეგს აგზავნის პლატფორმაში, API გასაღებით (Pre-QC hosted-ის მოდელი). ამ შემთხვევაში სერვერს REDI/WFS არ სჭირდება.
6. **თუ ვერ მოგვცემს:** Pre-QC ისე დარჩება, როგორც დღეს მუშაობს (ლოკალურად), და პლატფორმაში მხოლოდ შედეგები გამოჩნდება.
7. 🗣️ **მარტივად:** „ავტომატური წინასწარი შემოწმება დღეს ადამიანის კომპიუტერიდან მუშაობს, მისივე „საშვებით“. სერვერზე რომ გადავიდეს, სისტემისთვის განკუთვნილი „საშვი“ გვჭირდება. ეს მოგვიანებით ეტაპია.“

R18. თანამშრომლების მონაცემი (გუნდები, ახალი / წასული)

1. **რა გვჭირდება:**
   - **(ა)** პროცესი: HR/IT ჩვენს ადმინებს ატყობინებს ახალ, გადაყვანილ და წასულ თანამშრომლებზე;
   - **(ბ)** შემდეგ ეტაპზე, შესაძლოა, HR/AD-იდან წაკითხვა: სახელი, ელფოსტა, აქტიურობა, განყოფილება, 💡 პირადი ნომერი ცალსახა მიბმისთვის.
2. **რატომ:**
   - ✅ გუნდები დღეს Google ცხრილიდან (Field Group) მოდის Apps Script-ით;
   - ✅ პლატფორმას უკვე აქვს `teams` / `team_members` თარიღებით და ადმინისტრირების გვერდი;
   - ✅ გათიშვა (`active = false`) მყისიერად მოქმედებს, მაგრამ ჩვენ უნდა ვიცოდეთ ამის შესახებ. ⚠️ G18: გათიშვა გუნდის წევრობას ჯერ ავტომატურად არ ხურავს (ჩვენი გასასწორებელია).
3. **დამოკიდებულია:** მენეჯერის მიკუთვნება, წვდომის გაუქმება.
4. **S, P3.**
5. **ალტერნატივები:** ადმინები ხელით მართავენ (✅ შესაძლებელია დღესაც).

6. **თუ ვერ მოგვცემს:** რისკი: წასული თანამშრომელი აქტიური რჩება, სანამ ვინმე არ შეგვატყობინებს.
7. 🗣 **მარტივად:** „როცა ვინმე იწყებს მუშაობას, სხვა გუნდში გადადის ან მიდის, ჩვენც უნდა ვიცოდეთ, რომ წვდომა დროულად შევცვალოთ.“

R19. საიდუმლოებების შენახვა

1. **რა გვჭირდება:** ადგილი, სადაც სერვერზე ინახება:
   - `QCP_DATABASE_URL` (ბაზის პაროლით);
   - `QCP_SESSION_SECRET`;
   - `QCP_GOOGLE_CLIENT_SECRET`;
   - `QCP_SMTP_PASSWORD`;
   - მომავალში: წყაროების ანგარიშები. ეს შეიძლება იყოს გარემოს ფაილი `640` უფლებით, აპლიკაციის მომხმარებლისთვის, ან თქვენი საიდუმლოების საცავი (Vault / Credential Manager / Docker secrets).
2. **რატომ:**
   - ✅ კოდში საიდუმლოება არ წერია (`app/config.py`);
   - ⚠️ `QCP_SESSION_SECRET`-ის გარეშე გასაღები `data/.session_secret`-ში იქმნება. საწარმოოში ცვლადით უნდა მიეთითოს.
3. **დამოკიდებულია:** სესიები, ბაზა, Google, SMTP.
4. **M, P1.**
5. **ალტერნატივები:** —
6. **თუ ვერ მოგვცემს:** —
7. 🗣 **მარტივად:** „პაროლები სერვერზე, „სეიფში“. ჩვენ მათი ნახვაც არ გვჭირდება: შეგიძლიათ, პირდაპირ თქვენ ჩაწეროთ.“

R20. მონიტორინგი, ლოგები, განგაში

1. **რა გვჭირდება:**
   - ✅ health check: `GET /api/health` (1 წუთში ერთხელ);
   - VM-ის მეტრიკები: CPU, RAM, დისკი;
   - PostgreSQL-ის მეტრიკები: კავშირები, ზომა, ნელი შეკითხვები;
   - proxy-ს წვდომის ლოგი (query string-ის გარეშე);
   - აპლიკაციის ლოგი;
   - განგაში: სერვისი გაჩერდა, დისკი > 80%, ბექაპი ჩავარდა, 💡 ღამის იმპორტი ჩავარდა.
2. **რატომ:**
   - ⚠️ G16: დღეს აპლიკაციის ლოგი მინიმალურია;
   - ⚠️ `/api/health` ბაზის მიუწვდომლობისას 500-ს აბრუნებს. 💡 გავასწორებთ, თუ თქვენი health check ამას ითხოვს.
3. **დამოკიდებულია:** ინციდენტის დროული აღმოჩენა.
4. **S (P1), M (P3).**
5. **ალტერნატივები:** მარტივი uptime-შემოწმება + ლოგები ფაილში.

6. **თუ ვერ მოგვცემს:** პრობლემას მომხმარებლები შეგვატყობინებენ, ჩვენ კი მათზე გვიან გავიგებთ.
7. 🗣 **მარტივად:** „სერვერს „პულსს“ ყოველ წუთს ვუმოწმებთ. თუ გაჩერდა ან დისკი გაივსო, ვიღაცას უნდა ეცნობოს.“

R21. სატესტო გარემო (staging)

1. **რა გვჭირდება:** მეორე, მცირე გარემო: VM ან კონტეინერი + ცალკე ბაზა. მასზე მოწმდება განახლებები და მიგრაციები, და ტარდება დატვირთვის ტესტი. მონაცემი: ანონიმიზებული ან საწარმოოს ასლი, თქვენი წესით. ცალკე Google redirect URI.
2. **რატომ:** ⚠️ G1/G2 — PostgreSQL-ზე გადასვლა და მიგრაციები ჯერ გამოსაცდელია; დატვირთვის ტესტი (Q13) საწარმოოზე არ ტარდება.
3. **დამოკიდებულია:** უსაფრთხო განახლებები.
4. **S, P1.**
5. **ალტერნატივები:** სატესტო ბაზა საწარმოო VM-ზე (ნაკლებად იზოლირებულია).
6. **თუ ვერ მოგვცემს:** განახლებები პირდაპირ საწარმოოში მოგვიწევს. ეს რისკია.
7. 🗣 **მარტივად:** „„სარეპეტიციო დარბაზი“, სადაც ახალ ვერსიას ჯერ იქ ვამოწმებთ და მერე ვაყენებთ.“

R22. ვერსიის კონტროლი და deployment-ის წვდომა

1. **რა გვჭირდება:**
   - git-ის რეპოზიტორია: სააგენტოს GitLab / Azure DevOps / Gitea, ან private საცავი თქვენი წესით;
   - deployment-ის გზა: ან SSH/RDP წვდომა ჩვენთვის (შეზღუდული, აპლიკაციის საქაღალდეზე), ან CI/CD პაიპლაინი, ან განახლება თქვენი ადმინისტრატორის მიერ, ჩვენი პროცედურით.
2. **რატომ:**
   - ⚠️ G17: პროექტი ჯერ git-ში არ არის;
   - IT-ს უნდა ხედავდეს, რა იცვლება;
   - დაბრუნება ვერსიით.
3. **დამოკიდებულია:** განახლებები, მხარდაჭერა, ცოდნის გადაცემა (Q8).
4. **M, P1.**
5. **ალტერნატივები:** —
6. **თუ ვერ მოგვცემს:** კოდი ჩვენს კომპიუტერზე დარჩება, და ეს მხარდაჭერის რისკია.
7. 🗣 **მარტივად:** „კოდის „არქივი“ გვჭირდება, სადაც ყოველი ცვლილება ჩანს და უკან დაბრუნება შეიძლება. სასურველია, თქვენთან, რომ თქვენც ხედავდეთ.“

R23. უსაფრთხოების შემოწმება, პერსონალური მონაცემი, შენახვის ვადები

1. **რა გვჭირდება:**
   - **(ა)** უსაფრთხოების შემოწმება ან პენტესტი საწარმოომდე (თქვენი სტანდარტით);

    - **(ბ)** პერსონალური მონაცემის დამუშავების შეფასება და სამართლებრივი საფუძველი, განსაკუთრებით თანამშრომლის პირადი ნომრის შენახვაზე (✅ გადაწყვეტილებით ინახება ვინაობის შესადარებლად, ნიღბით ჩანს);
    - **(გ)** შენახვის ვადები: QC ჩანაწერები, აუდიტი, მეილები.
2. **რატომ:**
    - ✅ მონაცემის მინიმიზაცია: მოქალაქეების პირადი ნომრები არ იკითხება;
    - ✅ როლები, 404, აუდიტი;
    - ⚠️ ცნობილი ხარვეზები: G3 (დემო შესვლა), G8 (CSRF token), G9 (`/docs`), ფაილის ტიპის შემოწმება შიგთავსით. ყველა გასწორდება საწარმოომდე.
3. **დამოკიდებულია:** საწარმოო გაშვების ნებართვა.
4. **M, P3.**
5. **ალტერნატივები:** —
6. **თუ ვერ მოგვცემს:** სამართლებრივი რისკი.
7. 🗣 **მარტივად:** „სანამ ყველა თანამშრომელი შევა, გვინდა, რომ თქვენმა უსაფრთხოებამ და იურისტებმა შეამოწმონ: რას ვინახავთ, რამდენ ხანს, და კარგად არის თუ არა დაცული.“

R24. დრო და სასაათო სარტყელი

1. **რა გვჭირდება:** NTP სინქრონი; სერვერის სასაათო სარტყელი **Asia/Tbilisi**, ან შეთანხმება, რომ სერვერი UTC-ზეა და ჩვენ კოდს შევცვლით.
2. **რატომ:** ⚠️ G10: დრო `datetime.now()`-ით, სარტყლის გარეშე იწერება. D (შემოწმების დრო), „დღის რაოდენობა“ და სტატისტიკის პერიოდები სერვერის ადგილობრივ დროზეა დამოკიდებული. UTC-ზე მყოფი სერვერი 4 საათით დაძრავდა თარიღებს.
3. **დამოკიდებულია:** სტატისტიკა, დღის რაოდენობა, ისტორიის დროები.
4. **M, P1.**
5. **ალტერნატივები:** 💡 კოდის შეცვლა: შენახვა UTC-ში, ჩვენება Asia/Tbilisi-ში. ეს უფრო სწორი გადაწყვეტაა, მაგრამ მიგრაციამდე უნდა გაკეთდეს.
6. **თუ ვერ მოგვცემს:** არასწორი თარიღები სტატისტიკაში.
7. 🗣 **მარტივად:** „სერვერის საათი თბილისის დროზე უნდა იყოს. თორემ ღამის 11-ზე გაგზავნილი ოქმი „ხვალინდელად“ ჩაითვლება.“

R25. ოპერაციული პასუხისმგებლობა (RACI) და ინციდენტები

1. **რა გვჭირდება:** წერილობითი შეთანხმება:

სფერო	პასუხისმგებელი (💡 შეთავაზება)
VM, OS, პაჩები, ქსელი, firewall, სერტიფიკატი	IT
PostgreSQL-ის ექსპლუატაცია, ბექაპი, აღდგენის ტესტი	IT (DBA)
აპლიკაციის განახლება	ჩვენ ვამზადებთ; ერთად ვაყენებთ

სფერო	პასუხისმგებელი (💡 შეთავაზება)
აპლიკაციის ადმინისტრირება (მომხმარებლები, როლები, გუნდები, განაწილება)	QC ადმინები: ლუკა ნუკრაძე, სალომე დავითაშვილი
შეცდომები კოდში	ჩვენ
ინციდენტი: სერვისი არ პასუხობს	IT (პირველი ხაზი) → ჩვენ
ინციდენტი: არასწორი მონაცემი / წესი	ჩვენ
უსაფრთხოების ინციდენტი	IT-უსაფრთხოება + ჩვენ

2. **რატომ:** ინციდენტის დროს უნდა ვიცოდეთ, ვინ რეკავს ვის.
3. **დამოკიდებულია:** მხარდაჭერა.
4. **M, P1.**
5. **ალტერნატივები:** —
6. **თუ ვერ მოგვცემს:** —
7. 🗣 **მარტივად:** „წინასწარ შევთანხმდეთ, ვინ რას უვლის. თქვენ — „შენობას“, ჩვენ — „მაღაზიას“.“

R26. ატვირთული ფაილების ანტივირუსული შემოწმება (არჩევითი)

1. **რა გვჭირდება:** ანტივირუსული შემოწმების სერვისი (მაგ. ClamAV ან თქვენი), რომელსაც აპლიკაცია ატვირთვისას გამოიძახებს.
2. **რატომ:**
   - ✅ ატვირთვა მხოლოდ QC როლებს შეუძლიათ, მხოლოდ სურათი ან PDF, ≤ 10 MB, შემთხვევითი სახელით;
   - ⚠️ ტიპი ბრაუზერის `content_type`-ით მოწმდება. 💡 შიგთავსის შემოწმებასაც დავამატებთ.
3. **დამოკიდებულია:** ფაილის უსაფრთხოება.
4. **O, P3.**
5. **ალტერნატივები:** ფაილის სერვერის ანტივირუსი (პერიოდული სკანირება).
6. **თუ ვერ მოგვცემს:** რისკი დაბალია: ტვირთავს მხოლოდ კონტროლი.
7. 🗣 **მარტივად:** „ფოტოებს მხოლოდ კონტროლი ტვირთავს, მაგრამ თუ ანტივირუსის სერვისი გაქვთ, სიამოვნებით ჩავრთავთ.“

---

3. ინფორმაცია, რომელიც IT-სგან უნდა მივიღოთ

#	კითხვა	ეხება
1	სად შეიძლება სერვერის განთავსება და რა OS / ვირტუალიზაცია / კონტეინერი გაქვთ სტანდარტად?	R1, R2

#	კითხვა	ეხება
2	PostgreSQL თუ SQL Server — რა არის სტანდარტი? გაქვთ DBA და არსებული კლასტერი?	R3
3	ბექაპის სისტემა: ინსტრუმენტი, შენახვის ვადა, PITR შესაძლებელია?	R4, R5
4	ქვედომენის სახელი და სერტიფიკატის წყარო (შიდა / საჯარო CA)?	R6, R7
5	Proxy / load balancer / WAF: თქვენი გამოვიყენოთ? მისი IP (`forwarded-allow-ips` -ისთვის)?	R8
6	გარე წვდომა: საჯარო HTTPS თუ VPN? რა დამატებითი დაცვა მოითხოვება?	R10
7	სერვერიდან გამავალი კავშირი Google-თან: პირდაპირ თუ proxy-ით? PyPI-ის სარკე გაქვთ?	R11
8	Google Workspace აქვს სააგენტოს? Google Cloud პროექტს ვინ შექმნის და ვის სახელზე?	R12
9	SMTP relay: host, პორტი, ავთენტიფიკაცია; გამგზავნი მისამართი; SPF/DKIM?	R13
10	REGFIELDALL: view შესაძლებელია? თუ არა — ექსპორტის ფორმატი, ადგილი, დრო? ვინ არის მონაცემის მფლობელი?	R14
11	სპორადულის სია: საიდან იქმნება? view ან SFTP შესაძლებელია, მხოლოდ საჭირო სვეტებით?	R15
12	ოქმის შინაარსის (შემსრულებლები, დანართები) API არსებობს? ფორმატი?	R16
13	REDI, SLR, WFS: სერვისის ანგარიში სერვერისთვის შესაძლებელია? IP-ის შეზღუდვა?	R17
14	HR/AD: ახალი და წასული თანამშრომლის შესახებ შეტყობინების პროცესი? ამზომველებს სააგენტოს ანგარიში აქვთ?	R18, Q45
15	საიდუმლოებების საცავი გაქვთ?	R19
16	მონიტორინგის სისტემა და ლოგების ფორმატი?	R20
17	სატესტო გარემო შესაძლებელია? მონაცემის წესი სატესტოზე?	R21
18	კოდის საცავი (git) და deployment-ის წესი; ჩვენი წვდომა სერვერზე?	R22
19	უსაფრთხოების შემოწმების და პერსონალური მონაცემის შეფასების პროცედურა; შენახვის ვადები?	R23
20	სერვერის სასაათო სარტყელი?	R24
21	მხარდაჭერის მოდელი, SLA, ინციდენტის არხი?	R25
22	VM-ის, ბაზის და დომენის გამოყოფის სავარაუდო ვადა?	ყველა

4. გადაწყვეტილებების ჟურნალი (შესავსები შეხვედრაზე)

#	გადაწყვეტილება	ვარიანტები	არჩეული	ვინ	თარიღი
D1	OS / გაშვების გარემო	Linux VM · Windows Server · Docker host · Kubernetes			
D2	ბაზის ძრავა	PostgreSQL · SQL Server			
D3	ბაზის მდებარეობა	იმავე VM-ზე · არსებულ კლასტერზე			
D4	გარე წვდომა	საჯარო HTTPS · VPN · მხოლოდ შიდა (პილოტი)			
D5	შესვლა	Google (Gmail) · Google Workspace (დომენით) · Entra ID / AD			
D6	REGFIELDALL-ის წყარო	view · API · ექსპორტი (GDB / CSV)			
D7	სპორადულის წყარო	view · SFTP · მეილი (როგორც დღეს)			
D8	Pre-QC-ის ინტეგრაცია	ოქმის API · ექსპორტი · Validator ლოკალურად + შედეგის API			
D9	ბექაპი / RPO / RTO	ღამის · PITR			
D10	deployment-ის მოდელი	ჩვენ (SSH) · IT · CI/CD			

---

5. რას ვაკეთებთ ჩვენ (ჩვენი ვალდებულებები IT-ის წინაშე)

ეს Handoff-ის §8.3-ის (G1–G18) შეჯამებაა: რას ვასწორებთ საწარმოომდე. ყველა 💡 დაგეგმილია და ჯერ არ არის გაკეთებული.

#	ჩვენი სამუშაო	რატომ
W1	PostgreSQL-ის დრაივერი + ტესტები PostgreSQL-ზე	G1
W2	Alembic მიგრაციები	G2
W3	საწარმოოს რეჟიმი: დემო შესვლა ფიზიკურად გათიშული; `Secure` cookie ყოველთვის; `--proxy-headers`	G3, G4
W4	ვერსიის შემოწმება ბაზის დონეზე	G5
W5	გადახედვა ბაზაში (რამდენიმე worker-ისთვის)	G6
W6	ფონური worker: ღამის იმპორტი (upsert), მეილის რიგი, გასუფთავება	G7, G11

#	ჩვენი სამუშაო	რატომ
W7	CSRF token; `/docs` მხოლოდ ადმინისთვის ან გათიშული; CSP	G8, G9
W8	დროის სარტყლის გადაწყვეტა	G10
W9	წყაროს კონექტორები IT-ის არჩეული გზისთვის	G12
W10	ფაილების საცავის კონფიგურაცია; შიგთავსის ტიპის შემოწმება	G13
W11	სტატისტიკის ოპტიმიზაცია PostgreSQL-ზე; დატვირთვის ტესტი	G14
W12	ელფოსტების შეგროვება და შეყვანა (მენეჯერებთან ერთად)	G15
W13	ლოგირება (query string-ის გარეშე), health `degraded` რეჟიმი	G16
W14	git, ვერსიები, ცვლილებების ჟურნალი	G17
W15	გათიშვისას გუნდის წევრობის დახურვა; მასობრივი გადაწერა	G18
W16	Dockerfile (თუ Docker აირჩევა); lock ფაილი ბიბლიოთეკებისთვის	A.11, E.4
W17	ოპერაციული დოკუმენტაცია IT-სთვის: ინსტალაცია, განახლება, აღდგენა	R25

🗣 **მარტივად:** „თქვენ რასაც გთხოვთ, იმის პარალელურად ჩვენც ვამზადებთ ჩვენს ნაწილს. ეს სიაა: რა უნდა შევცვალოთ, რომ ერთი კომპიუტერის პროტოტიპი სერვერისთვის მზად იყოს.“

---

6. რესურსის შეფასება — ვარაუდები ცალ-ცალკე

რა	✅ გაზომილი	💡 შეფასება	ვარაუდი
მომხმარებლები	605 ჩანაწერი ბაზაში; აქტიური როლი: 247 ამზომველი, 14 მენეჯერი, 14 ოპერატორი, 2 QCM, 2 ადმინი, 2 დემო	≈ 300 აქტიური, ≈ 40 ერთდროულად	`IT_REQUEST.md`-ის შეფასება
აპლიკაციის ბაზა	94 MB (SQLite) / 44 159 შემთხვევა	+0.5–2 GB წელიწადში	50–100 ათასი შემთხვევა წელიწადში; 5–10 მოვლენა თითოზე; PostgreSQL-ის ზომა SQLite-ზე მეტია
REGFIELDALL-ის ასლი	≈ 1 GB (11 ველი, 2.33 მლნ)	1–2 GB	იგივე ველები PostgreSQL-ში
ფოტოები	0	1–3 GB წელიწადში	`IT_REQUEST.md`

რა	✅ გაზომილი	💡 შეფასება	ვარაუდი
ბექაპი	—	ბაზის ზომა × შენახვის დღეები (შეკუმშვით ნაკლები)	14–30 დღე
CPU / RAM	ტესტები 6 წმ; სტატისტიკა 2–3 წმ (SQLite)	4 vCPU / 8 GB (ბაზით)	დატვირთვის ტესტი არ ყოფილა
ქსელი	—	მცირე: HTML გვერდები, ერთჯერადი ECharts ≈ 1 MB (ქეშდება), ფოტოები ≤ 10 MB	—

---

7. თანმიმდევრობა

1. **P1 — პილოტი:**
   - IT: R1, R2, R3, R4 (ღამის), R5, R6, R7, R8, R9, R11, R12, R19, R22, R24, R25;
   - ჩვენ: W1–W5, W7, W8, W12, W14;
   - მონაცემი დღევანდელი გზით მოდის (Distributor-ის ასლი, სპორადული მეილით);
   - მცირე ჯგუფი: QC + 1–2 საველე გუნდი.
2. **P2 — წყაროები:** R14, R15 (+R16 სურვილისამებრ); ჩვენ: W6, W9.
3. **P3 — სრული გაშვება:** R10, R13, R18, R20, R23 (+R26); ჩვენ: W10, W11, W13, W15–W17; ძველი ცხრილის მიგრაცია და cutover.
4. **P4 — Pre-QC:** R16, R17.

🗣 **მარტივად:** „ჯერ პატარა ჯგუფით ვიწყებთ, მინიმალური რესურსით. მერე ავტომატურ მონაცემს ვრთავთ, მერე ყველას, და ბოლოს ავტომატურ წინასწარ შემოწმებას.“